

5. Input/Output

- 5.1.Principles of I/O hardware
- 5.2.Principles of I/O software
- 5.3.Disks structure and scheduling
- 5.4.Clocks
- 5.5.Terminals

5. Input/Output

5.1. Principles of I/O hardware

One of the important jobs of an Operating System is to manage various I/O devices including mouse, keyboards, touch pad, disk drives, display adapters, USB devices, Bitmapped screen, LED, Analog-to-digital converter, On/off switch, network connections, audio I/O, printers etc.

An I/O system is required to take an application I/O request and send it to the physical device, then take whatever response comes back from the device and send it to the application.

5.1.1: I/O Devices

- Not much to say. Devices are varied.
- I/O devices can be divided into two categories:
- Block versus character devices:
 - Devices, such as disks and CDROMs, with addressable chunks (sectors in this case) are called **block** devices, These devices support seeking.
 - Devices, such as Ethernet and modem connections, that are a stream of characters are called **character** devices. These devices do not support seeking.
 - Some cases, like tapes, are not so clear.

5.1.2: Device Controllers

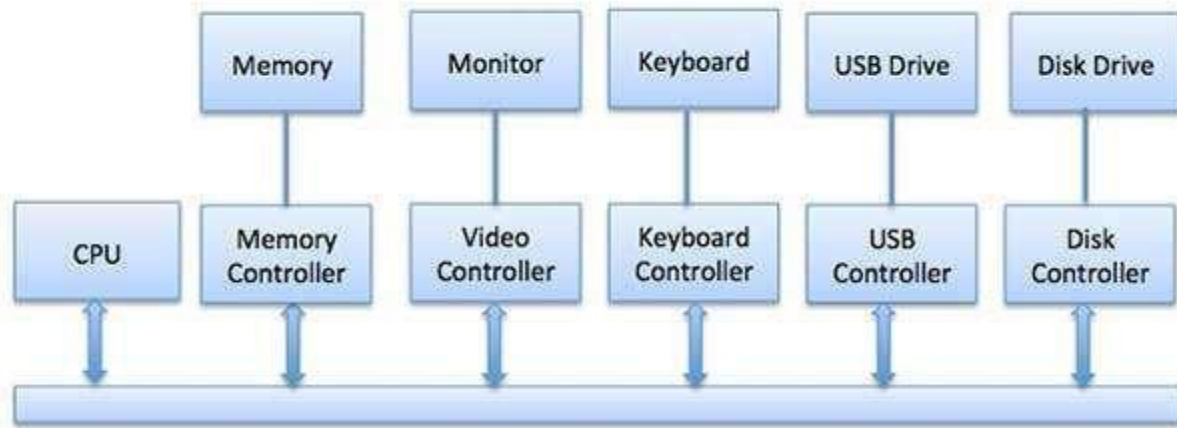
Device drivers are software modules that can be plugged into an OS to handle a particular device. Operating System takes help from device drivers to handle all I/O devices.

The Device Controller works like an interface between a device and a device driver. I/O units (Keyboard, mouse, printer, etc.) typically consist of a mechanical component and an electronic component where electronic component is called the device controller.

There is always a device controller and a device driver for each device to communicate with the Operating Systems. A device controller may be able to handle multiple devices. As an interface its main task is to convert serial bit stream to block of bytes, perform error correction as necessary.

Any device connected to the computer is connected by a plug and socket, and the socket is connected to a device controller. Following is a model for connecting the CPU, memory, controllers, and I/O devices where CPU and device controllers all use a common bus for communication.

These are the ``devices" as far as the OS is concerned. That is, the OS code is written with the controller specification in hand not with the device specification.



- Also called **adaptors**.
- The controller abstracts away some of the low level features of the device.
- For disks, the controller does error checking and buffering.
- Controllers are also called **adaptors**.

Synchronous vs Asynchronous I/O

- **Synchronous I/O** — In this scheme CPU execution waits while I/O proceeds
- **Asynchronous I/O** — I/O proceeds concurrently with CPU execution

Communication to I/O Devices

The CPU must have a way to pass information to and from an I/O device. There are three approaches available to communicate with the CPU and Device.

- Special Instruction I/O
- Memory-mapped I/O
- Direct memory access (DMA)

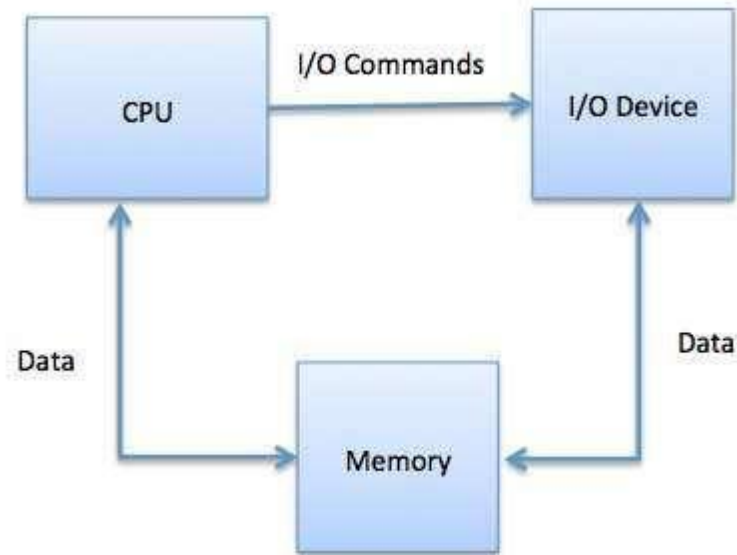
Special Instruction I/O

This uses CPU instructions that are specifically made for controlling I/O devices. These instructions typically allow data to be sent to an I/O device or read from an I/O device.

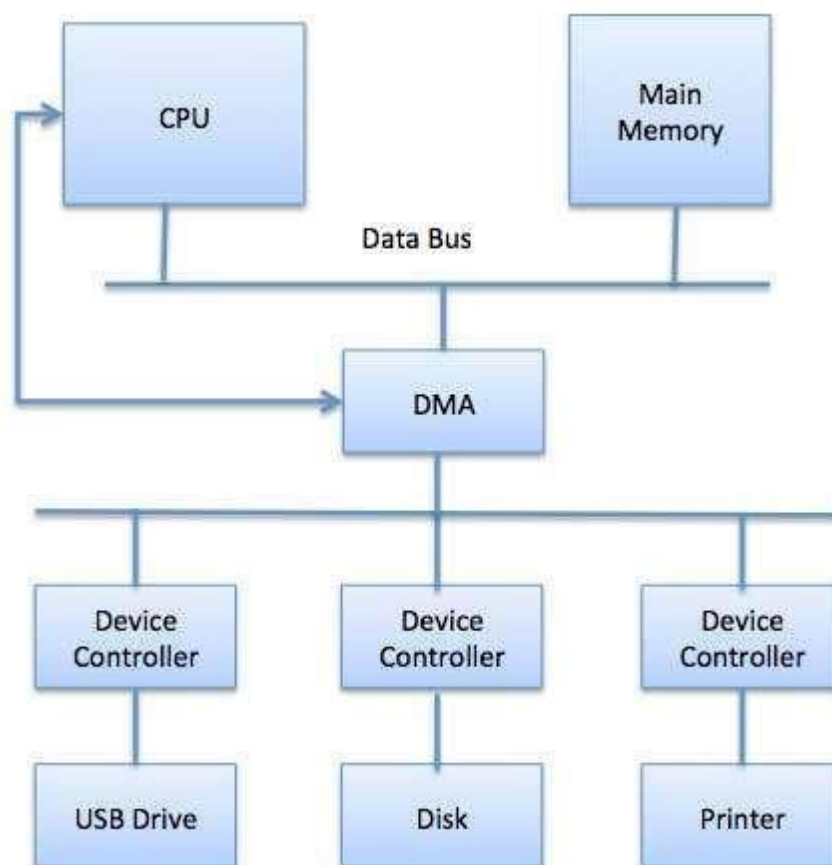
5.1.3: Memory-Mapped I/O

When using memory-mapped I/O, the same address space is shared by memory and I/O devices. The device is connected directly to certain main memory locations so that I/O device can transfer block of data to/from memory without going through CPU. While using memory mapped IO, OS allocates buffer in memory and informs I/O device to use that buffer to send data to the CPU. I/O device operates asynchronously with CPU, interrupts CPU when finished.

The advantage to this method is that every instruction which can access memory can be used to manipulate an I/O device. Memory mapped IO is used for most high-speed I/O devices like disks, communication interfaces.



5.1.4: Direct Memory Access (DMA)



Slow devices like keyboards will generate an interrupt to the main CPU after each byte is transferred. If a fast device such as a disk generated an interrupt for each byte, the operating system would spend most of its time handling these interrupts. So a typical computer uses direct memory access (DMA) hardware to reduce this overhead. Direct Memory Access (DMA) means CPU grants I/O module authority to read from or write to memory without involvement. DMA module itself controls exchange of data between main

memory and the I/O device. CPU is only involved at the beginning and end of the transfer and interrupted only after entire block has been transferred.

Direct Memory Access needs a special hardware called DMA controller (DMAC) that manages the data transfers and arbitrates access to the system bus. The controllers are programmed with source and destination pointers (where to read/write the data), counters to track the number of transferred bytes, and settings, which includes I/O and memory types, interrupts and states for the CPU cycles.

The operating system uses the DMA hardware as follows:

Step	Description
1	Device driver is instructed to transfer disk data to a buffer address X.
2	Device driver then instruct disk controller to transfer data to buffer.
3	Disk controller starts DMA transfer.
4	Disk controller sends each byte to DMA controller.
5	DMA controller transfers bytes to buffer, increases the memory address, decreases the counter C until C becomes zero.
6	When C becomes zero, DMA interrupts CPU to signal transfer completion.

Polling vs Interrupts I/O

A computer must have a way of detecting the arrival of any type of input. There are two ways that this can happen, known as **polling** and **interrupts**. Both of these techniques allow the processor to deal with events that can happen at any time and that are not related to the process it is currently running.

Polling I/O

Polling is the simplest way for an I/O device to communicate with the processor. The process of periodically checking status of the device to see if it is time for the next I/O operation, is called **polling**. The I/O device simply puts the information in a Status register, and the processor must come and get the information.

Most of the time, devices will not require attention and when one does it will have to wait until it is next interrogated by the polling program. This is an inefficient method and much of the processors time is wasted on unnecessary polls.

Compare this method to a teacher continually asking every student in a class, one after another, if they need help. Obviously the more efficient method would be for a student to inform the teacher whenever they require assistance.

Interrupts I/O

An alternative scheme for dealing with I/O is the interrupt-driven method. An interrupt is a signal to the microprocessor from a device that requires attention.

A device controller puts an interrupt signal on the bus when it needs CPU's attention when CPU receives an interrupt, It saves its current state and invokes the appropriate interrupt handler using the interrupt vector (addresses of OS routines to handle various events).

When the interrupting device has been deal with, the CPU continues with its original task as if it had never been interrupted.

5.2. Principles of I/O Software

A key concept in the design of I/O software is that it should be device independent where it should be possible to write programs that can access any I/O device without having to specify the device in advance.

For example, a program that reads a file as an input should be able to read a file on a floppy disk, on a hard disk, or on a CDROM, without having to modify the program for each different device.

Device Drivers

Device drivers are software modules that can be plugged into an OS to handle a particular device. Operating System takes help from device drivers to handle all I/O devices.

Device driver is generally written by the device's manufacturer and delivered along with the device on a CD-ROM.

A device driver performs the following jobs –

- To accept request from the device independent software above to it.
- Interact with the device controller to take and give I/O and perform required error handling.
- Making sure that the request is executed successfully.

Interrupt handlers

An interrupt handler, also known as an *Interrupt Service Routine* or ISR, is a piece of software or more specifically a callback function in an operating system or more specifically in a device driver, whose execution is triggered by the reception(process of receiving something sent) of an interrupt.

When the interrupt happens, the interrupt procedure does whatever it has to in order to handle the interrupt, updates data structures and wakes up process that was waiting for an interrupt to happen.

Device-Independent I/O Software

The basic function of the device-independent software is to perform the I/O functions that are common to all devices and to provide a uniform interface to the user-level software. Though it is difficult to write completely device independent software but we can write some modules which are common among all the devices.

Following is a list of functions of device-independent I/O Software –

- Uniform interfacing for device drivers
- Device naming - Mnemonic names mapped to Major and Minor device numbers

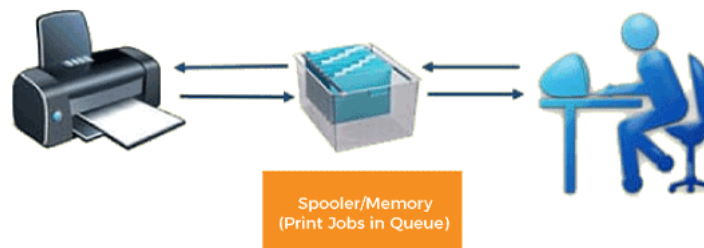
- Device protection
- Providing a device-independent block size
- Buffering because data coming off a device cannot be stored in final destination.
- Storage allocation on block devices
- Allocation and releasing dedicated devices
- Error Reporting

User-Space I/O Software

These are the libraries which provide richer and simplified interface to access the functionality of the kernel or ultimately interactive with the device drivers.

Most of the user-level I/O software consists of library procedures with some exception like spooling system which is a way of dealing with dedicated I/O devices in a multiprogramming system.

Simultaneous Peripheral Operation Online [Spooling] refers to a technique used in computing and data processing to manage input and output operations. It allows you to send a job to a device, such as a printer, while continuing to work on other tasks without waiting for the operation to be completed.



I/O Libraries (e.g., stdio) are in user-space to provide an interface to the OS resident device-independent I/O S/W. For example putchar(), getchar(), printf() and scanf() are example of user level I/O library stdio available in C programming.

Disks Structure And Scheduling

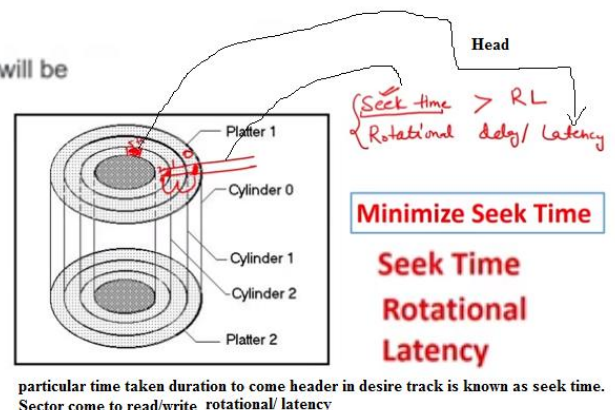
Disk Scheduling

Disk scheduling is the method of scheduling the I/O requests arriving for the disk.

Disk scheduling Algorithms tells the order in which I/O requests will be selected by disk controller to fulfill the Read Write requests

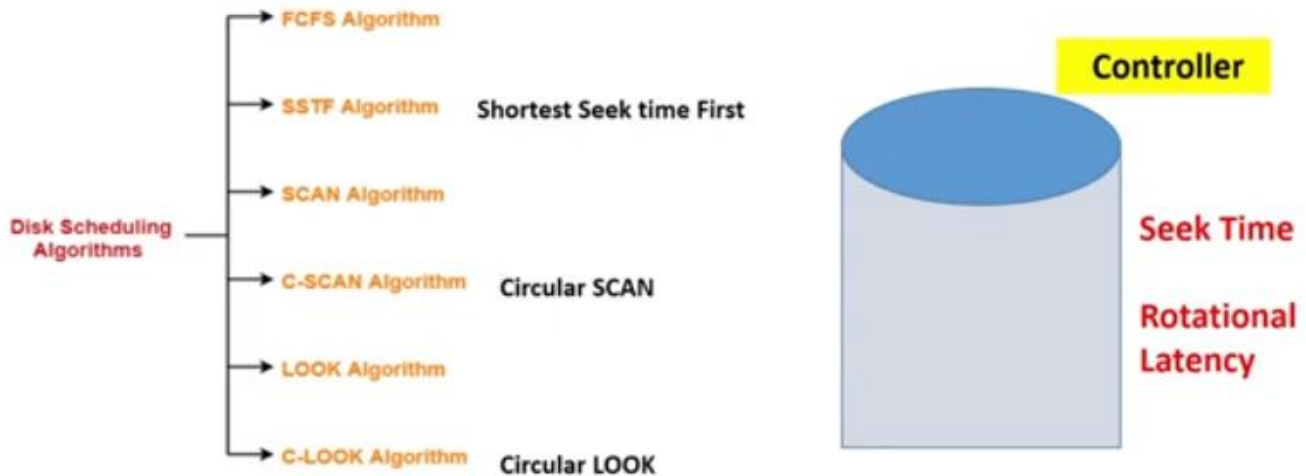
The operating system is responsible for using hardware efficiently — for the disk drives, this means having a **fast access time** and **disk bandwidth**.

Access time = Seek time + Rotational Delay + other overhead
Disk bandwidth = total bytes transferred / time



Disk Scheduling Algorithms

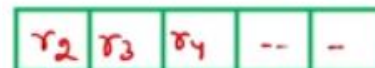
When one request is completed, the operating system chooses which pending request to service next. How does the operating system make this choice? Any one of several disk-scheduling algorithms can be used:



Disk Scheduling

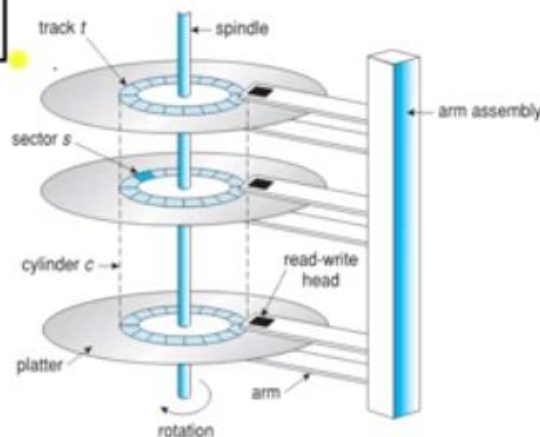
Format of I/O Requests:

Request



<Platter no, Surface no, cylinder no, sector no >

- Read or Write Opⁿ
- no. of bytes to read/Write



Controller

Seek Time
Rotational Latency

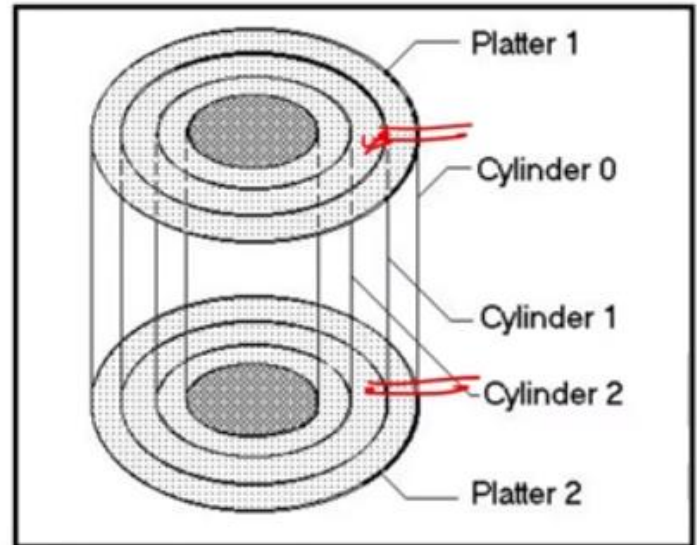
How Disk Scheduling Numericals are asked ?

Request Queue containing cylinder no:

2	183	3	122	14
---	-----	---	-----	----

Initial RW Head Position = 1st cylinder

Total RW head Movements or Total Seeks = ?



Disks Structure

Disks come in many sizes and speeds, and information may be stored optically or magnetically; however, all disks share a number of important features.

For Example: floppy disks, hard disks, CD-ROMs and DVDs. Disk surface is divided into number of logical blocks called *sectors* and *tracks*.

The term *cylinder* refers to all the tracks at particular head position in hard disk.

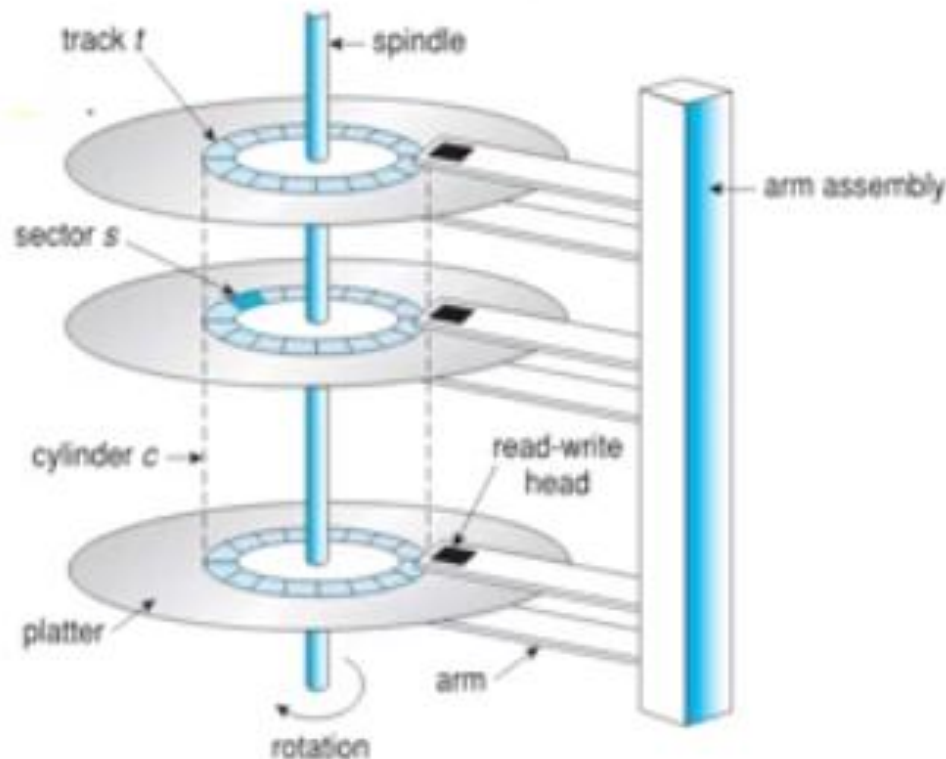


Fig: Disk Structure

Disk Operations

Latency(delay) Time: The time taken to rotate from its current position to a position adjacent to the read-write head.

Seek(the time required by the read/write head to move from one track to another): The processes of moving the arm assembly to new cylinder.

To access a particular record, first the arm assembly must be moved to the appropriate cylinder, and then rotate the disk until it is immediately under the read-write head.

The time taken to access the whole record is called *transmission time*.

Disk Scheduling

OS is responsible to use the hardware efficiently – for the disk drive this means fast seek, latency and transmission time.

For most disks, the *seek time* dominates the other two times, so reducing the *mean seek time* can improve system performance significantly.

First-Come First-Served (FCFS)

The first request to arrive is the first one serviced.

Advantages: Simple and Fair.

Problems: Does not provide fastest service.

Example:

FCFS Disk Scheduling Algorithm

A Hard Disk has total 200 cylinders

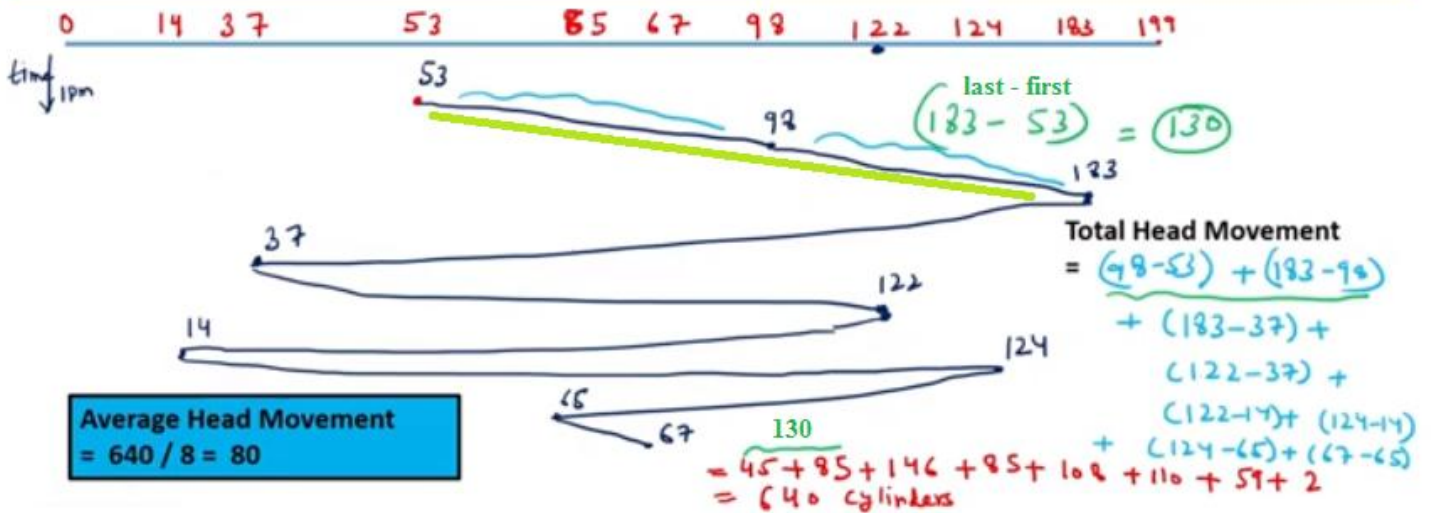
Request Queue (cylinders):

98, 183, 37, 122, 14, 124, 65, 67

Initial Position of Read/write head = 53th cylinder

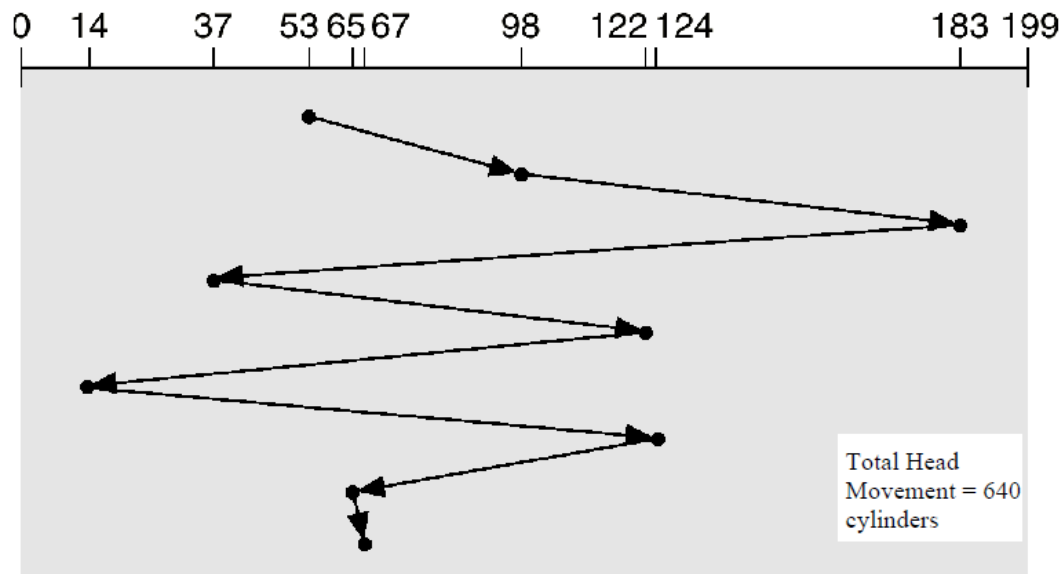
1. Find Total Head Movements / Total seeks/ Total Cylinders Traversed

2. Find Average Head Movements = ?



queue = 98, 183, 37, 122, 14, 124, 65, 67

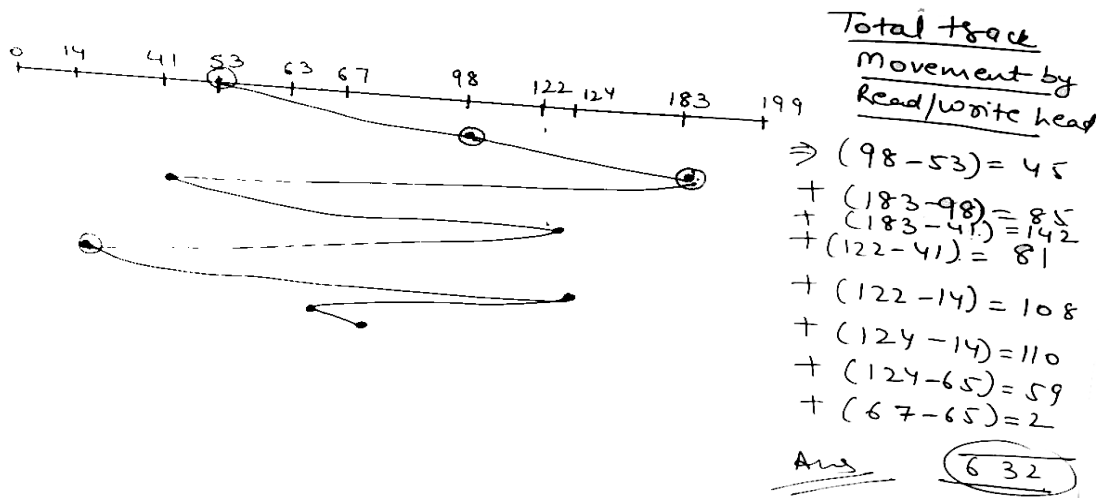
head starts at 53



Disk Scheduling Algorithm... FIFO (First come first serve)

Track request $\rightarrow 98, 183, 41, 122, 14, 124, 65, 67$

Read/write head is at $\rightarrow 53$



Shortest-Seek-Time-First (SSTF)

Selects the request with the minimum seek time from the current head position.

Advantages: Gives a substantial improvement in performance.

Problems: SSTF scheduling is a form of SJF scheduling; may cause starvation of some requests. Not optimal.

Used in batch system where throughput is the major consideration but unacceptable in interactive system.

SSTF Disk Scheduling Algorithm

A Hard Disk has 200 cylinders

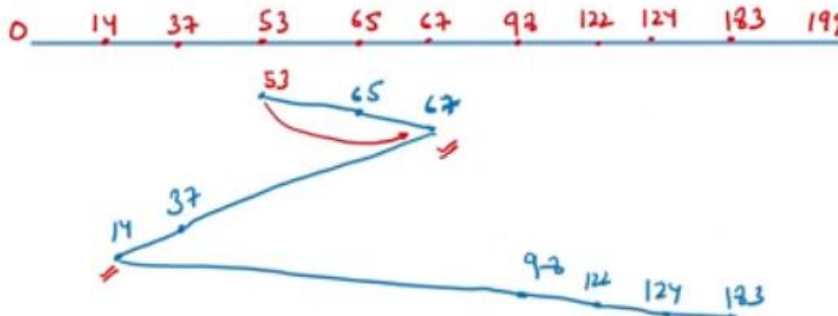
Request Queue (cylinders):

98, 183, 37, 122, 14, 124, 65, 67

Initial Position of Read/write head = 53th cylinder

1. Find Total Head Movements / Total seeks/ Total Cylinders Traversed

2. Find Average Head Movements = ?



Total Head Movement

$$= (67 - 53) + (67 - 14) + (183 - 14)$$

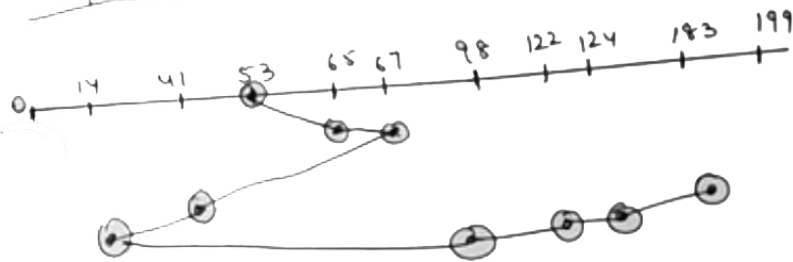
$$= 14 + 53 + 169$$

$$= 236 \text{ cylinders}$$

Average Head Movement

$$= 236 / 8 = 29.5$$

SSTF (Shortest seek time first)
 ack request $\rightarrow$ 98, 183, 41, 122, 14, 124, 65, 67
 Read/write head is at $\rightarrow$ 53



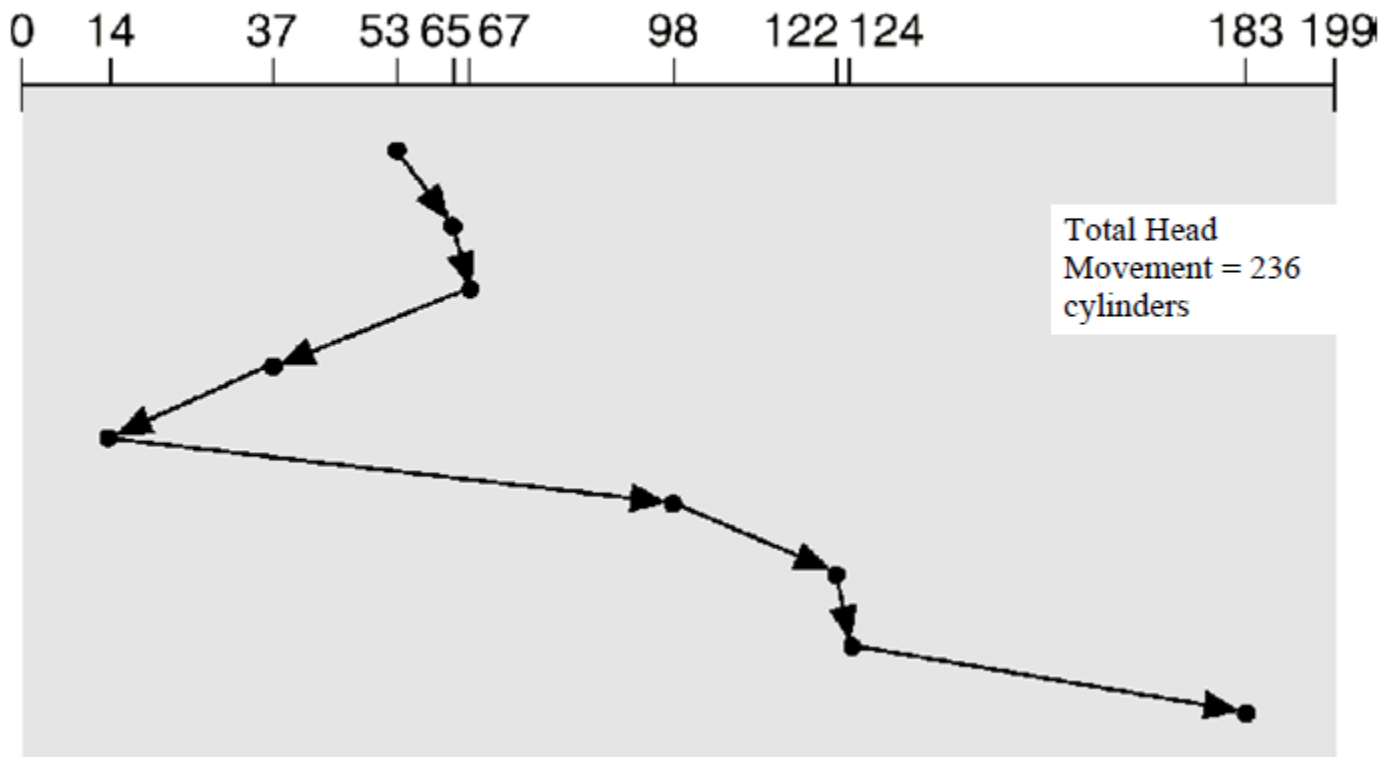
Total Track Movement

$$\begin{aligned}
 &= (65 - 53) = 12 \\
 &+ (67 - 65) = 2 \\
 &+ (67 - 41) = 26 \\
 &+ (41 - 14) = 27 \\
 &+ (98 - 14) = 84 \\
 &+ (122 - 98) = 24 \\
 &+ (124 - 122) = 2 \\
 &+ (183 - 124) = 59
 \end{aligned}$$

Ans

236

queue = 98, 183, 37, 122, 14, 124, 65, 67
 head starts at 53



Total Head Movement = 236 cylinders

SCAN

The disk arm starts at one end of the disk, and moves toward the **other end**, servicing requests until it gets to the **other end** of the disk, where the head movement is reversed and servicing continues. Sometimes called the *Elevator Algorithm*. (lift)

(request in middle gets advantage)

Before applying SCAN to schedule the requests, we must know

1. head's current position.
2. Direction of head movement in which it is moving

Advantages: Decreases variances in seek and improve response time.

Problem: Starvation is possible if there are repeated request in current track.

Question:

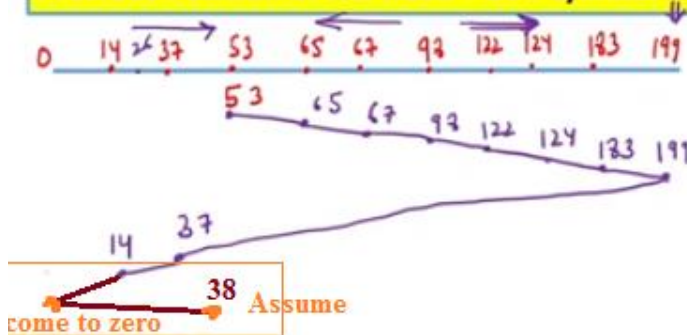
A Hard Disk has total 200 cylinders
Request Queue (cylinders):
98, 183, 37, 122, 14, 124, 65, 67
Initial Position of Read/write head = 53th cylinder
Previous Position of RW head = 26th cylinder

1. Find Total Head Movements / Total seeks/ Total Cylinders Traversed =?
2. Find Average Head Movements = ?

SCAN (or Elevator Algo)

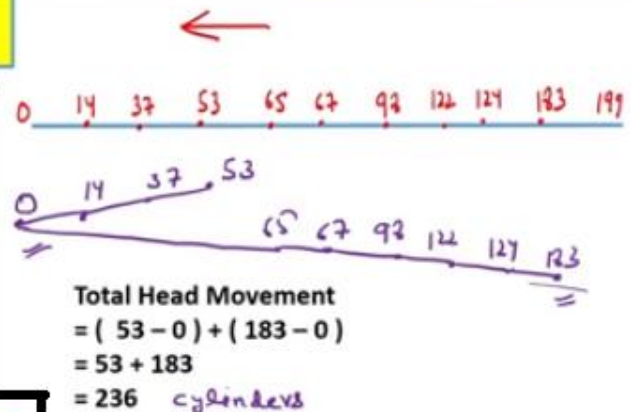
A Hard Disk has total 200 cylinders 0 – 199
Request Queue (cylinders):
98, 183, 37, 122, 14, 124, 65, 67
Initial Position of Read/write head = 53th cylinder
Previous Position of RW head = 26th cylinder

1. Find Total Head Movements / Total seeks/ Total Cylinders Traversed =?
2. Find Average Head Movements = ?



Total Head Movement
= (199 – 53) + (199 – 14)
= 146 + 185
= 331 cylinders

Average Head Movement
= $331/8 = 41.375$



AHM = $236/8 = 29.5$

C-SCAN (modified version of SCAN Algorithm)

When a uniform distribution of request for cylinders, only the few request are in extreme cylinders, since these cylinders have recently been serviced.

Why not go to the next extreme?

Circular SCAN is a variant of SCAN designed to provide a more uniform wait time.

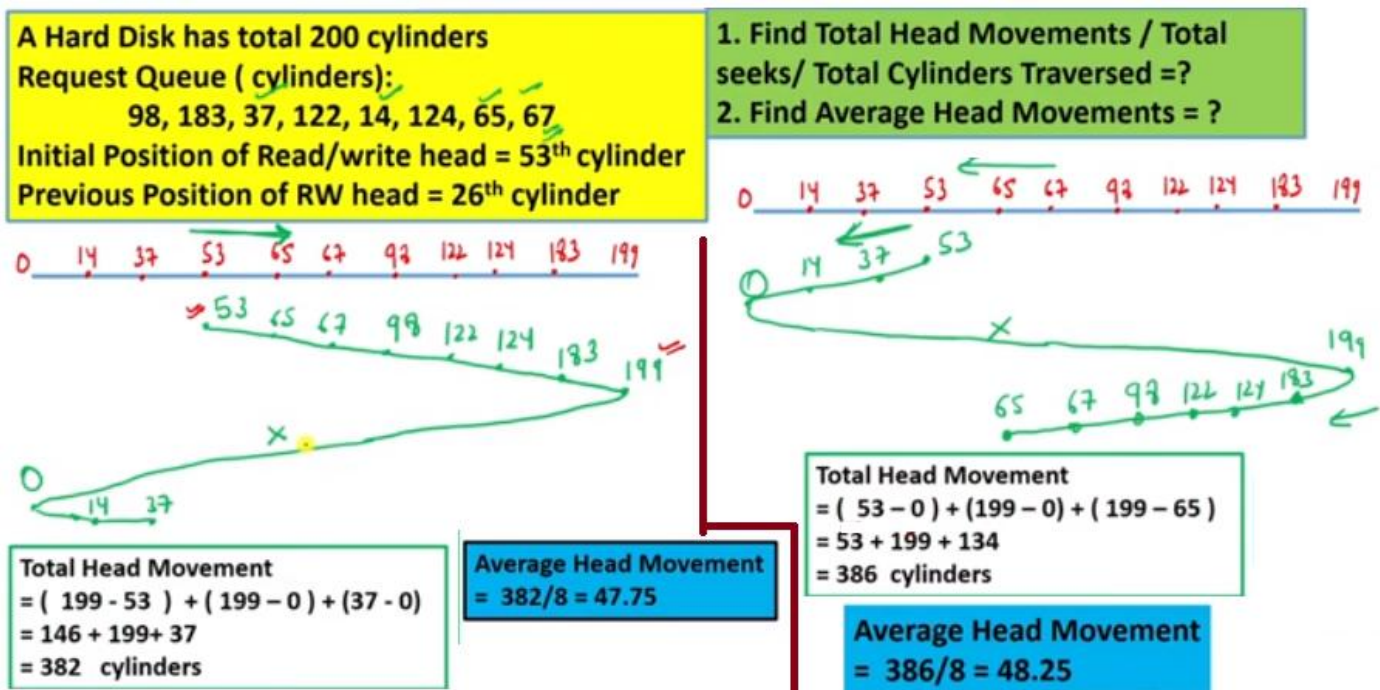
The head moves from one end of the disk to the other. Servicing requests as it goes like SCAN Algorithm.

But

When it reaches the other end, however, it immediately returns to the beginning of the disk, **without servicing any requests on the return trip.**

Treats the cylinders as a circular list that wraps around from the last cylinder to the first one.

**C-SCAN =
SCAN + No servicing of request in return trip (back Journey)**



Why C-SCAN is called Circular Elevator Algorithm ?

It treats the cylinders as a **circular list** that wraps around from the final cylinder to the first one.

Summary: C-SCAN Algorithm

C-SCAN =

SCAN + No servicing of request in Round trip Journey(back Journey)

Head moves only in Single Direction unlike SCAN algo.

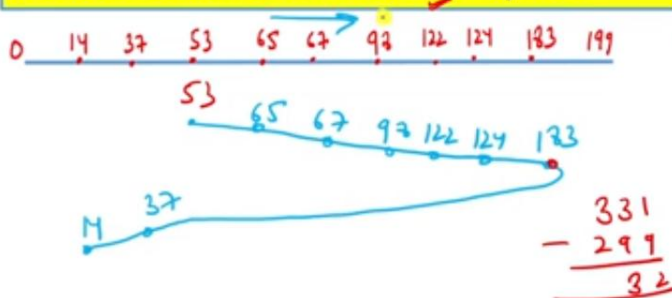
Provide uniform Waiting time for all the I/O requests unlike SCAN algo.

LOOK Disk Scheduling Algorithm

Improved Version of SCAN Algorithm

LOOK = SCAN + Go till only the last request present in I/O Request Queue (Do not go till the end cylinders)

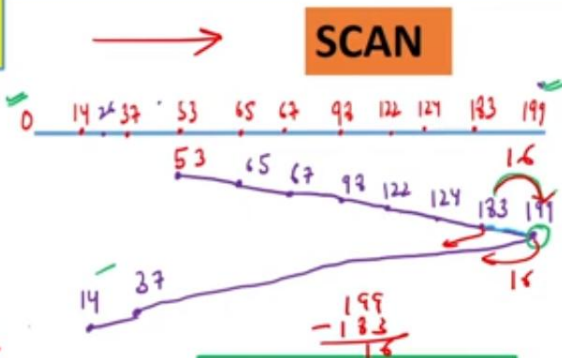
A Hard Disk has total 200 cylinders
Request Queue (cylinders):
98, 183, 37, 122, 14, 124, 65, 67
Initial Position of Read/write head = 53th cylinder
Previous Position of RW head = 26th cylinder



Total Head Movement
= (183 - 53) + (183 - 14)
= 130 + 169
= 299 cylinders

Average Head Movement
= 299/8 = 37.375

1. Find Total Head Movements / Total seeks/ Total Cylinders Traversed = ?
2. Find Average Head Movements = ?



Total Head Movement
= (199 - 53) + (199 - 14)
= 146 + 185
= 331 cylinders

LOOK Disk Scheduling Algorithm

A Hard Disk has total 200 cylinders

Request Queue (cylinders):

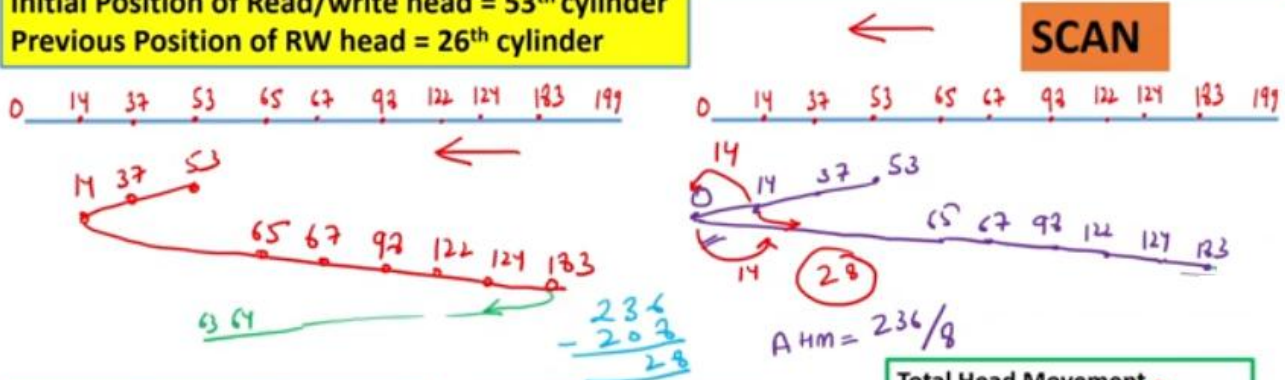
98, 183, 37, 122, 14, 124, 65, 67, 64, 63

Initial Position of Read/write head = 53th cylinder

Previous Position of RW head = 26th cylinder

1. Find Total Head Movements / Total seeks/ Total Cylinders Traversed = ?

2. Find Average Head Movements = ?



LOOK Disk Scheduling Algorithm

1. Look is more efficient than SCAN

It improves efficiency by avoiding unnecessary seek operation

2. Gives High Throughput than SCAN

LOOK completes requests in less time than SCAN

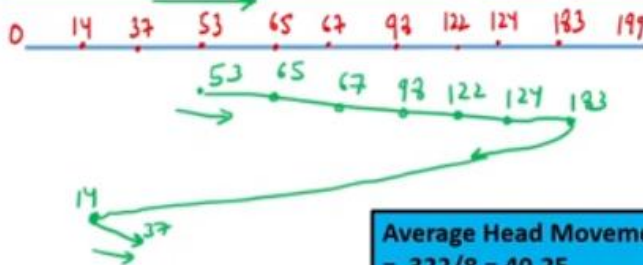
C-LOOK Disk Scheduling Algorithm

Modified Version of LOOK & SCAN Algorithm

C - LOOK = LOOK + SCAN + No servicing of request in Round trip Journey(back Journey)

C - LOOK Disk Scheduling Algorithm

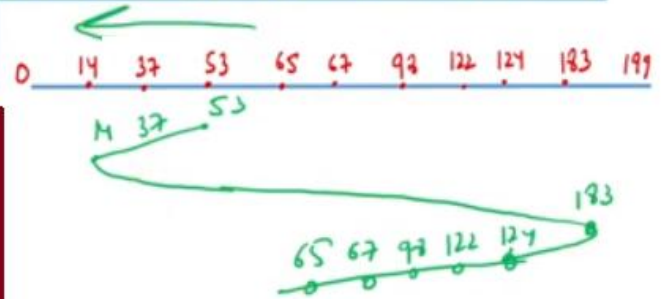
A Hard Disk has total 200 cylinders
Request Queue (cylinders):
98, 183, 37, 122, 14, 124, 65, 67
Initial Position of Read/write head = 53th cylinder
Previous Position of RW head = 26th cylinder



Average Head Movement
= $322/8 = 40.25$

Total Head Movement
= $(183 - 53) + (183 - 14) + (37 - 14)$
= $130 + 169 + 23$
= 322 cylinders

1. Find Total Head Movements / Total seeks/ Total Cylinders Traversed = ?
2. Find Average Head Movements = ?



Total Head Movement
= $(53 - 14) + (183 - 14) + (183 - 65)$
= $39 + 169 + 118$
= 326 cylinders

Average Head Movement
= $326/8 = 40.75$

LOOK vs C-LOOK Disk Scheduling Algorithm

LOOK	C - LOOK
1. LOOK serves requests in both directions	1. C - LOOK serves requests in one direction only
2. Higher Throughput than C - LOOK	2. Lesser Throughput than LOOK
3. Waiting time & response time of requests is non- uniform	3. Uniform Waiting time & response time of requests

https://www.youtube.com/watch?v=vvJbznPCajI&list=PLVKIC9j3jSYtCEAffM_51qVWeQus70cqc&index=32

https://www.youtube.com/watch?v=GAfvd54am1U&list=PLVKIC9j3jSYtCEAffM_51qVWeQus70cqc&index=40

<https://www.youtube.com/watch?v=YduCK0MPjFI>

<https://cs.nyu.edu/~gottlieb/courses/2000s/2002-03-fall/os/lectures/lecture-13.html>
<https://www.studocu.com/vn/document/truong-dai-hoc-fpt/operating-system/os-part-05/63991680>

<https://www.csd.uoc.gr/~hy345/notes/pdf/5.pdf>

<https://www.arjun00.com.np/2023/07/operating-system-solution-dcom-it-ctevt.html>
https://mrcet.com/downloads/digital_notes/CSE/II%20Year/OPERATING%20SYSTEMS%20%20NOTES%20R18.pdf
https://www.google.com/search?q=diploma+OS+note&oq=diploma+OS+note&gs_lcrp=EgZjaHJvbWUyBggAEEUYOdIBCTExMzYyYajBqN6gCD7ACAQ&client=ms-android-xiaomi-terr1-rso2&sourceid=chrome-mobile&ie=UTF-8

<https://www.youtube.com/watch?v=7DoP1L9nAAs>

FCFS

<https://www.youtube.com/watch?v=0hSmekhy4xI>

Shortest-Seek-Time-First (SSTF)

<https://www.youtube.com/watch?v=0XCfcWbPLSU>
<https://www.youtube.com/watch?v=0XCfcWbPLSU>
https://www.youtube.com/watch?v=dEt7mr9R_Z8